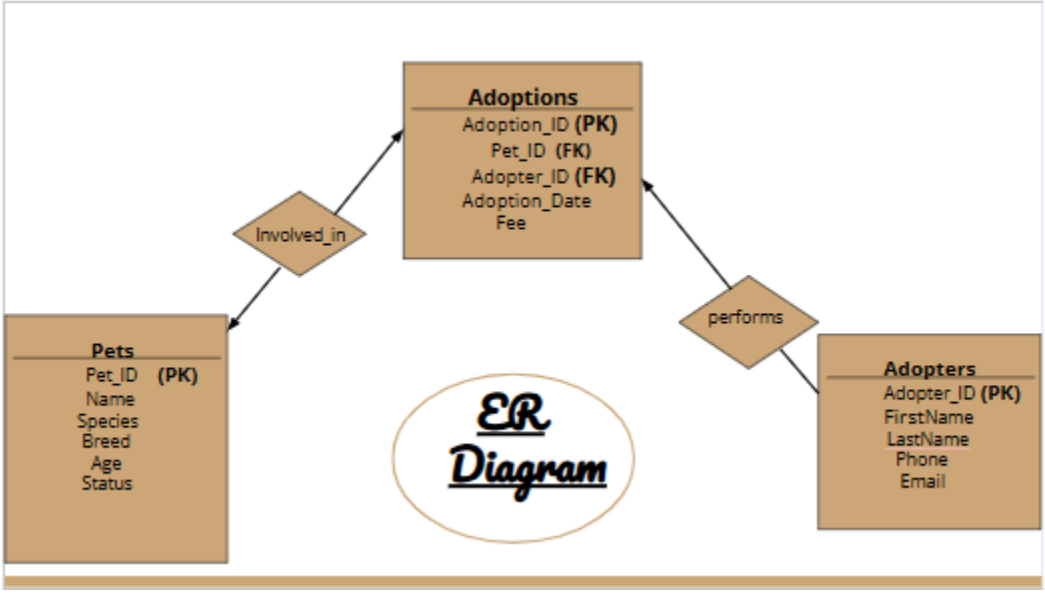
Lavisha Nigam
3798628
CS-1103

Final Report

Pet Connect

ER Diagram:-



DDL Statements for each entity:-

```
CREATE TABLE Pets (
   Pet_ID INTEGER PRIMARY KEY,
   Name TEXT,
   Species TEXT,
   Breed TEXT,
   Age INTEGER,
   Status TEXT
);

CREATE TABLE Adopters (
   Adopter_ID INTEGER PRIMARY KEY,
   FirstName TEXT,
   LastName TEXT,
   Phone TEXT,
```

```
  Email TEXT
);
```

DDL Statements for each relationships :-

```
CREATE TABLE Adoptions (
  Adoption_ID INTEGER PRIMARY KEY,
  Pet_ID INTEGER,
  Adopter_ID INTEGER,
  Adoption_Date TEXT,
  Fee REAL,
  FOREIGN KEY (Pet_ID) REFERENCES Pets(Pet_ID),
  FOREIGN KEY (Adopter_ID) REFERENCES Adopters(Adopter_ID)
);
```

Explanation of Entity DDLs :-

- **Mapping Entities to Tables**: The `Pets` and `Adopters` tables represent the Entity Sets from the ER diagram. Each table stores specific objects (animals and people) that exist independently in the system.
- **Primary Keys**: In the SQL code, `Pet_ID` and `Adopter_ID` are defined as `PRIMARY KEY`. This corresponds to the underlined attributes in the ER diagram, ensuring every record is unique and can be identified without confusion.
- **Attributes**: The columns such as `Name`, `Species`, `Phone`, and `Email` match the list in the rectangles in the diagram, providing the necessary details for each entity.

Explanation of Relationship DDL (The Adoptions Table) :-

- **Mapping the Diamond**: The `Adoptions` table is the implementation of the **Relationship Set** (the diamond) from the ER diagram. It defines how a specific Pet and a specific Adopter are connected.
- **Referential Integrity (Foreign Keys)**: The `FOREIGN KEY` constraints for `Pet_ID` and `Adopter_ID` link this table back to the main entities. This ensures that an adoption can only be recorded if the pet and adopter already exist in the database, maintaining the logic shown by the lines in the diagram.
- **Relationship Attributes**: The `Adoption_Date` and `Fee` columns represent **Attributes on a Relationship Set**. These are placed in the `Adoptions` table rather than the `Pets` or `Adopters` tables because a "Fee" only exists when an adoption actually happens.

Explanation of Cardinality :-

The database is structured so that one `Adopter_ID` can appear multiple times in the `Adoptions` table which is called a one–to-many relationship, but each `Pet_ID` is intended to be unique to one adoption. This mirrors the directed arrow (one) and undirected line (many) notation used in the lecture slides which is called a one-to-one relationship.

Sample Insert Statements :-

INSERT INTO Adopters (FirstName, LastName, Phone, Email) VALUES ('Lavisha', 'Nigam', '506-555-1234', 'lavishanigam@gmail.com');

INSERT INTO Adopters (FirstName, LastName, Phone, Email) VALUES ('Junior', 'French', '506-487-8967', 'junior@Yahoo.com');

INSERT INTO Adopters (FirstName, LastName, Phone, Email) VALUES ('Sarthak', 'Sharma', '506-555-8888', 'sarthak@unb.ca');

INSERT INTO Adopters (FirstName, LastName, Phone, Email) VALUES ('Aarush', 'Gupta', '506-555-7777', 'aarushgupt@gmail.com');

INSERT INTO Pets (Name, Species, Breed, Age, Status) VALUES ('Cooper', 'Dog', 'Golden Retriever', 2, 'Available');

INSERT INTO Pets (Name, Species, Breed, Age, Status) VALUES ('Misty', 'Cat', 'Siamese', 3, 'Available');

INSERT INTO Pets (Name, Species, Breed, Age, Status) VALUES ('Luna', 'Dog', 'Husky', 1, 'Pending');

INSERT INTO Pets (Name, Species, Breed, Age, Status) VALUES ('Oliver', 'Cat', 'Tabby', 5, 'Adopted');

INSERT INTO Pets (Name, Species, Breed, Age, Status) VALUES ('Bella', 'Dog', 'Beagle', 3, 'Available');

INSERT INTO Pets (Name, Species, Breed, Age, Status) VALUES ('Simba', 'Cat', 'Maine Coon', 2, 'Adopted');

INSERT INTO Adoptions (Pet_ID, Adopter_ID, Adoption_Date, Fee) VALUES (1, 1, '2026-03-15', 50.00)

INSERT INTO Adoptions (Pet_ID, Adopter_ID, Adoption_Date, Fee) VALUES (2, 2, '2026-03-10', 75.00);

INSERT INTO Adoptions (Pet_ID, Adopter_ID, Adoption_Date, Fee) VALUES (4, 3, '2026-03-12', 60.00);

Sample Select Statements :-

SELECT * FROM Pets;

SELECT * FROM Adopters;

SELECT * FROM Adoptions;

SELECT * FROM Pets WHERE Status = ‘Available’;

SELECT * FROM Pets WHERE Breed = ‘Dog’ and Status = ‘Available’;

SELECT * FROM Pets WHERE Breed = ‘Cat’ and Status = ‘Available’;

SELECT * FROM Pets WHERE Breed = ‘Reptile’ and Status = ‘Available’;

SELECT * FROM Pets WHERE Breed = ‘Amphibian’ and Status = ‘Available’;

SELECT * FROM Pets WHERE Breed = ‘Small Mammal’ and Status = ‘Available’;

SELECT * FROM Adopters WHERE Adopter_ID = 1;

SELECT * FROM Adoptions WHERE Adoption_Date = ‘2026-03-10’;

Sample Update Statements :-

UPDATE Pets SET Status = ‘Adopted’ WHERE Pet_ID = 6;

UPDATE Pets SET Status = ‘Pending’ WHERE Pet_ID = 3;

UPDATE Pets SET Status = ‘Available’ WHERE Pet_ID =14 ;

<u>Sample Delete Statements</u> :-

DELETE FROM Pets WHERE Pet_ID = 9;

DELETS FROM Pets WHERE Pet_ID = 16;

Design the Program

- **Menus and Functions**: The application is a console-based system that uses a `Scanner` to interact with the user. The main menu allows users to search for pets, while the `searchAndBook()` function handles the core logic of querying the database and allowing a user to select a Pet ID for booking.
- **SQL Statements and Testing**: To ensure the program is sturdy, I implemented flexible SQL queries, allowing for searches by both species and breed. I tested these statements in the terminal to verify they return the correct records from the SQLite database. Some statements are :-

- SELECT * FROM Pets WHERE Breed LIKE 'Igua%' OR Breed LIKE 'Snake%';

Pet_ID	Name	Species	Breed	Age	Status
17	Iggy	Reptile	Iguana	3	Available

- SELECT * FROM Pets WHERE (Species = 'Reptile' OR Breed = 'Iguana') AND Status = ‘Available’;

Pet_ID	Name	Species	Breed	Age	Status
17	Iggy	Reptile	Iguana	3	Available

- SELECT Pets.Name, Pets.Species, Adoptions.Adoption_Date, Adoptions.Fee FROM Pets INNER JOIN Adoptions ON Pets.Pet_ID = Adoptions.Pet_ID;

Name	Species	Adoption_Date	Fee
Buddy	Dog	2026-03-15	50.0
Misty	Cat	2026-03-12	60.0

- SELECT Name, Species, Adoption_Date FROM Pets NATURAL JOIN Adoptions;

- SELECT Adopters.FirstName, Adopters.LastName, Pets.Name AS PetName, Adoptions.Adoption_Date FROM Adoptions JOIN Adopters ON Adoptions.Adopter_ID =Adopters.Adopter_ID JOIN Pets ON Adoptions.Pet_ID = Pets.Pet_ID;

FirstName	LastName	PetName	Adoption_Date
Lavisha	Nigam	Buddy	2026-03-15
Sarthak	Sharma	Misty	2026-03-12

Class Diagram :-

Java Program :-

DatabaseManager - PetAdoptionSystem

Class Edit Tools Options

DatabaseManager

Compile Undo Cut Copy Paste Find... Close Source Code

```java
/**
 * Driver Class
 *
 * @author Lavisha Nigam
 * @version Project Work
 */

import java.util.Scanner;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;
import java.sql.ResultSet;
import java.sql.PreparedStatement;

public class DatabaseManager
{
    private static final String DB_URL = "jdbc:sqlite:PetConnect.db" ;

    //Crreating tables if the file is empty
    public static void createTables() {
    // 1. Create Pets Table
    String sqlPets = "CREATE TABLE IF NOT EXISTS Pets (" +
                     "Pet_ID INTEGER PRIMARY KEY, " +
                     "Name TEXT, Species TEXT, Breed TEXT, " +
                     "Age INTEGER, Status TEXT);";

    // 2. Create Adopters Table
    String sqlAdopters = "CREATE TABLE IF NOT EXISTS Adopters (" +
                         "Adopter_ID INTEGER PRIMARY KEY, " +
                         "FirstName TEXT, LastName TEXT, " +
                         "Phone TEXT, Email TEXT);";

    // 3. Create Adoptions Table (Created last because of Foreign Keys)
    String sqlAdoptions = "CREATE TABLE IF NOT EXISTS Adoptions (" +
                          "Adoption_ID INTEGER PRIMARY KEY, " +
                          "Pet_ID INTEGER, Adopter_ID INTEGER, " +
                          "Adoption_Date TEXT, Fee REAL, " +
                          "FOREIGN KEY (Pet_ID) REFERENCES Pets(Pet_ID), " +
                          "FOREIGN KEY (Adopter_ID) REFERENCES Adopters(Adopter_ID));";

    try (Connection conn = DriverManager.getConnection(DB_URL);
         Statement stmt = conn.createStatement()) {
```

```java
        // Execute all three commands
        stmt.execute(sqlPets);
        stmt.execute(sqlAdopters);
        stmt.execute(sqlAdoptions);

        System.out.println("All tables (Pets, Adopters, Adoptions) checked/created.");

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

    //To show all pets (SELECT)
    public static void showAllPets() {
        String sql = "SELECT * FROM Pets";

        try(Connection conn = DriverManager.getConnection(DB_URL);
             Statement stmt = conn.createStatement()){

                ResultSet rs = stmt.executeQuery(sql);

                while (rs.next()) {
                    System.out.println(rs.getString("Name"));
                  }
               } catch (SQLException e) {
                    e.printStackTrace();
                }
               }

    // Method to Add a new pet (Insert)
    public static void addPet(String name, String species, String breed, int age, String status){
        String sql = "INSERT INTO Pets( Name, Species, Breed, Age, Status) VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = DriverManager.getConnection(DB_URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, name);
            pstmt.setString(2, species);
            pstmt.setString(3, breed);
            pstmt.setInt(4, age);
            pstmt.setString(5, status);
            pstmt.executeUpdate();
            System.out.println(name + " added successfully!");

        } catch (SQLException e) {
```

```java
        } catch (SQLException e) {
            e.printStackTrace();
        }

    }
    //Method to update a pet's status (Update)
    public static void updatePetStatus(int id, String newStatus) {
        String sql = "UPDATE Pets SET Status = ? WHERE Pet_ID = ?";
        try (Connection conn = DriverManager.getConnection(DB_URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
             pstmt.setString(1, newStatus);
              pstmt.setInt(2, id);
               pstmt.executeUpdate();
               System.out.println("Pet_ID " + id + " updated.");

    } catch (SQLException e ) {
        e.printStackTrace();
    }
}
    //Method to delete a pet
    public static void deletePet(int id) {
        String sql = "DELETE FROM Pets WHERE Pet_ID = ?";
         try (Connection conn = DriverManager.getConnection(DB_URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
             pstmt.setInt(1, id); //This targets the specific id number
             pstmt.executeUpdate();
             System.out.println("Pet_ID " + id + " deleted successfully!");

            }catch (SQLException e) {
                e.printStackTrace();
            }
    }

    public static void main(String[]args) {
        createTables(); //checking the schema
        deletePet(9);                                       //Sample statement for deleting lily
        addPet("Lily", "Cat", "Persian", 2, "Available"); //and then adding her back

        addPet("Iggy", "Reptile", "Iguana", 3, "Available");  // Reptiles (The "Iguana" suggestion)
        addPet("Slinky", "Reptile", "Ball Python", 2, "Available");

        addPet("Jumper", "Amphibian", "Bullfrog", 1, "Available"); // Amphibians
        addPet("Axel", "Amphibian", "Axolotl", 1, "Available");
```

```java
        addPet("Cheeks", "Small Mammal", "Hamster", 1, "Available");  // Small Mammals
        addPet("Shadow", "Small Mammal", "Sugar Glider", 2, "Available");
        showAllPets(); // Show the schema results (Select)

        }

        public static void searchAndBook() {
    Scanner input = new Scanner(System.in);

    System.out.println("--- Welcome to PetConnect Search ---");
    System.out.print("Enter a species (e.g. Reptile) or a breed (e.g. Iguana): ");
    String userInput = input.nextLine();

    // The updated SQL query that checks TWO columns so that they can search either the breed or the species
    // e.g. if they search reptile or iguana they will get iguana in the list

    String searchSql = "SELECT * FROM Pets WHERE (Species = ? OR Breed = ?) AND Status = 'Available'";

    try (Connection conn = DriverManager.getConnection(DB_URL);
         PreparedStatement pstmt = conn.prepareStatement(searchSql)) {

        // setting user's input for both question marks
        pstmt.setString(1, userInput);
        pstmt.setString(2, userInput);

        ResultSet rs = pstmt.executeQuery();

        System.out.println("\nResults for \"" + userInput + "\":");
        boolean found = false;
        while (rs.next()) {
            found = true;
            int id = rs.getInt("Pet_ID");
            String name = rs.getString("Name");
            String species = rs.getString("Species");
            String breed = rs.getString("Breed");

            System.out.println("ID: " + id + " | Name: " + name + " | " + species + " (" + breed + ")");
        }

        if (!found) {
            System.out.println("Sorry, no matches found for \"" + userInput + "\" right now.");
        } else {
            // Booking Logic
```

```java
            System.out.print("\nEnter the ID of the pet you'd like to book a visit for: ");
            int idToBook = input.nextInt();

            // Re-using updatePetStatus method
            updatePetStatus(idToBook, "Pending");
            System.out.println("Success! Visit booked. Pet status updated to Pending.");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
```

[I Have also uploaded the file of my java program and the SQLite file in the Code folder in Github]